Department of Electrical and Information Engineering

University of Ruhuna

Faculty of Engineering

Assignment 02 — Semester 7

Cloud Computing

EC7205

April 2026

HealthSync

A Scalable, Secured, and Highly Available
Cloud-Native Healthcare Platform

Group Members : EG/2021/4735 — Piyumal Ranasinghe
EG/20XX/XXXX — Member Two
EG/20XX/XXXX — Member Three
EG/20XX/XXXX — Member Four

Date : April 2026

1. Introduction

HealthSync is a cloud-native healthcare platform connecting patients with doctors through an event-driven microservices architecture deployed entirely on **Microsoft Azure**. The system addresses real-world healthcare challenges: appointment booking, prescription management, secure patient records, real-time email notifications, and integrated Stripe payments.
The platform comprises **7 microservices** (6 Node.js + 1 Python), a **React 18 SPA** frontend, and **21 Azure resources**—all provisioned via **13 reusable Terraform modules** and deployed through **GitHub Actions** CI/CD pipelines.

Technology Stack:

- **Frontend:** React 18, Tailwind CSS, Vite
- **API Gateway:** Express.js, JWT, Helmet
- **Backend:** Node.js 20 / Python 3.11
- **Databases:** Cosmos DB, PostgreSQL 16, Redis 6
- **Messaging:** Azure Service Bus (3 queues)
- **Email:** Azure Communication Services
- **Compute:** Azure Container Apps
- **CDN:** Azure Front Door Standard
- **IaC:** Terraform $\geq$1.9 (azurerm ~4.0)
- **CI/CD:** GitHub Actions (2 pipelines)

Repositories:

- Application: https://github.com/PiyumalKK/HealthSync
- Infrastructure (IaC): https://github.com/PixelTech-Solutions/HealthSync-Infra
- Terraform Reusable Template: https://github.com/PixelTech-Solutions/Terraform

2. Architecture

Figure 1 shows the complete architecture. Traffic enters through **Azure Front Door** (CDN + WAF), routing static requests (`/*`) to Blob Storage and API calls (`/api/*`) to the Gateway Container App. The gateway authenticates via JWT and proxies to six internal microservices. Async events flow through Service Bus queues to the always-on Notification Service.

2.1. Key Architecture Decisions

Table 1: Architecture decisions mapped to cloud computing principles

Decision	Rationale	Principle
7 Microservices	Independent scaling, deployment, fault isolation per domain	Scalability
API Gateway	Single entry for JWT auth, rate limiting, CORS, routing	Security
Database-per-service	Polyglot persistence (Cosmos + PG + Redis); no shared coupling	Loose coupling
Service Bus (3 queues)	Decouple producers from consumers; messages survive failures	Availability
Container Apps	Serverless auto-scale 0→3; pay only for consumption	Elasticity
Key Vault + Managed ID	Zero secrets in code; passwordless Azure-to-Azure auth	Security
Terraform (13 modules)	Reproducible, version-controlled infrastructure	DevOps/IaC

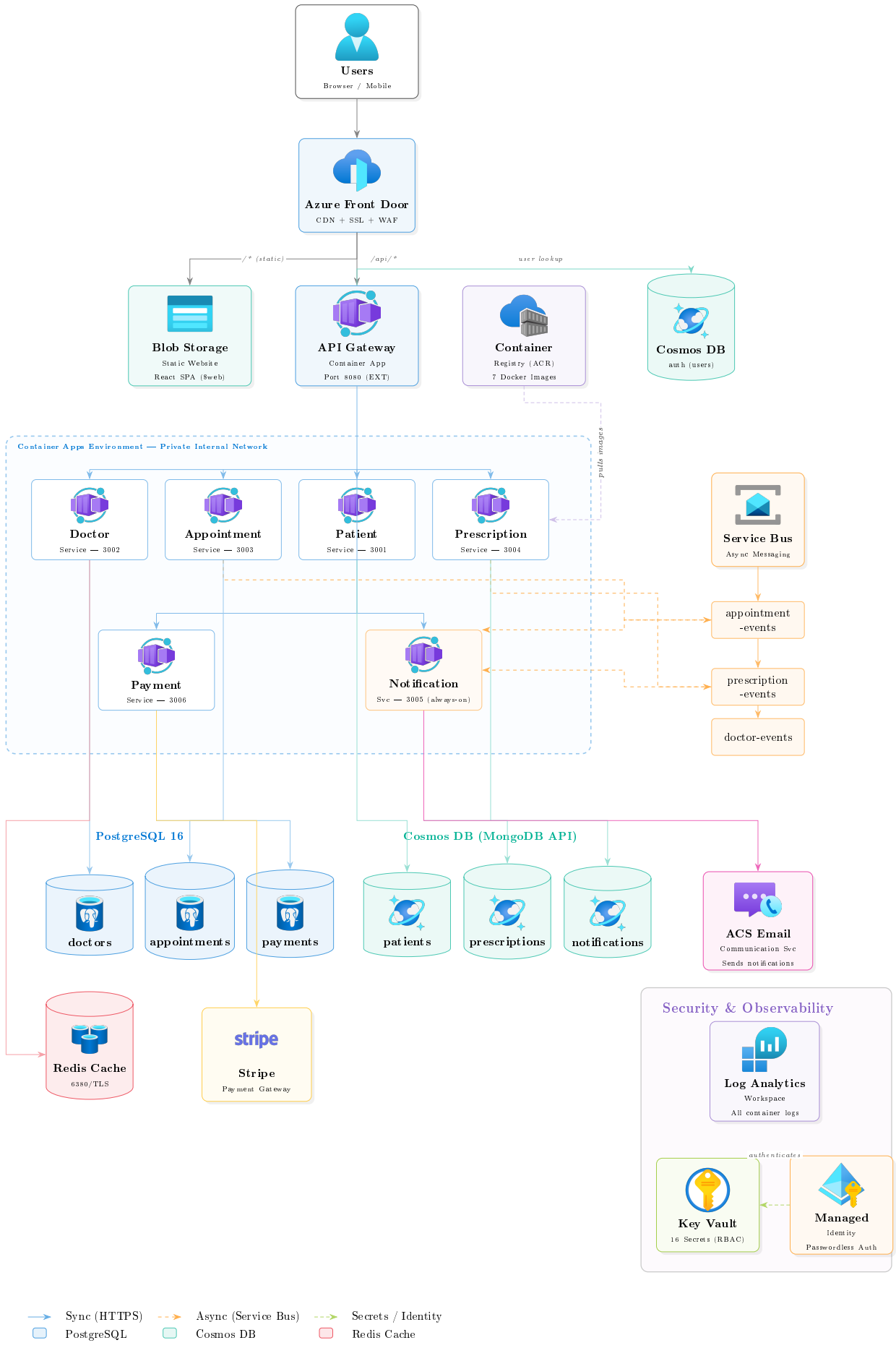

Figure 1: HealthSync complete cloud architecture — 7 microservices, 21 Azure resources, fully provisioned via Terraform IaC.

2.2. Communication Methods

Synchronous (HTTPS): Browser → Front Door → API Gateway → internal services for real-time request/response.
Asynchronous (Service Bus): Events published to queues (`appointment-events`, `prescription-even` `doctor-events`); the Notification Service (`min_replicas=1`) consumes and sends email via ACS. Booking returns in <200ms while notifications process asynchronously.

3. Implementation Steps

3.1. Infrastructure as Code (Terraform)

All 21 Azure resources are defined as **13 reusable Terraform modules** (resource-group, acr, container-app×7, container-apps-env, managed-identity, storage-account, key-vault, front-door, communication-services, cosmos-db, postgresql, redis, service-bus) composed in a service layer. Infrastructure is **never provisioned manually**—all changes flow through CI/CD.
Remote state: Terraform state is stored in Azure Blob Storage (`stpixeltechstate/tfstate`) with state locking via Azure Blob leases, enabling safe team collaboration without state conflicts.
Key Terraform features used:

- `dynamic` blocks — variable-length secret/env-var injection into Container Apps
- `lifecycle { ignore_changes = [image] }` — prevents CI/CD image updates from causing Terraform drift
- `depends_on` — ensures correct resource ordering (e.g., Key Vault before Container Apps)
- `sensitive = true` — masks secret values in plan output and logs

3.2. CI/CD Pipelines

Infrastructure pipeline (`infra-dev.yml`) calls a reusable Terraform template (org-wide) via `workflow_call`. `TF_VAR_*` GitHub Secrets inject sensitive values. GitHub Environments provide manual approval gates.

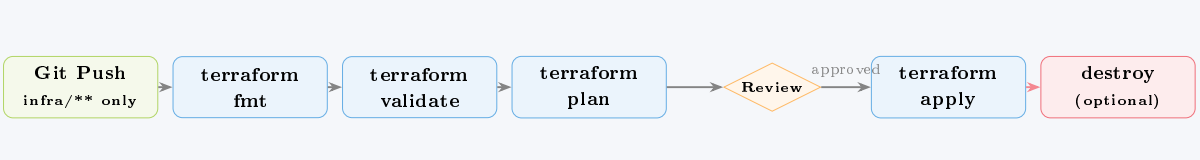

Figure 2: Infrastructure pipeline (reusable template): format → validate → plan → manual approval → apply.

Application pipeline (`deploy.yml`) uses manual dispatch with per-service selection. Docker images tagged with Git SHA for traceability and rollback.

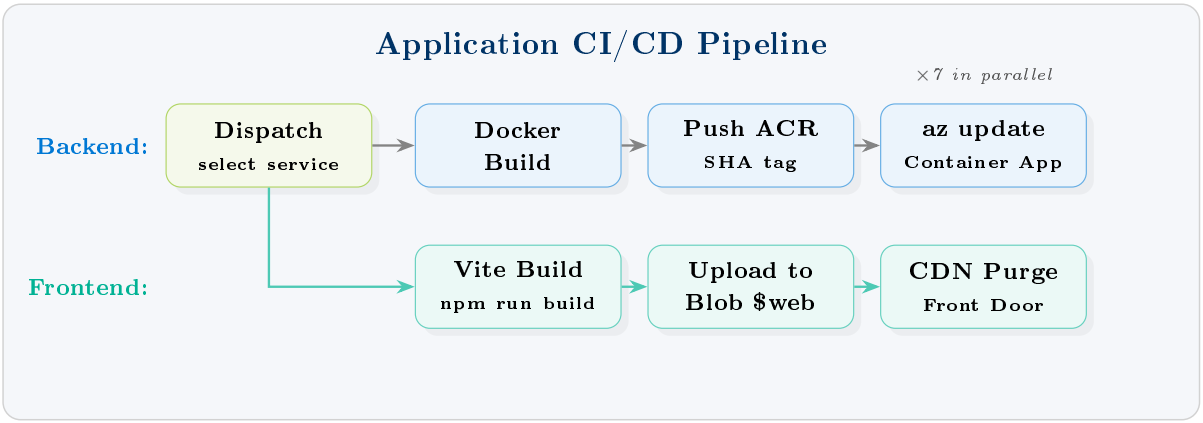

Figure 3: Application pipeline: per-service Docker build, ACR push, Container App update. Frontend uploads to Blob Storage.

3.3. Security Implementation (Five Layers)

1. **Network isolation:** Only API Gateway externally accessible; 6 services use private Container Apps DNS.
2. **Authentication:** JWT access + httpOnly refresh tokens, signed with Key Vault secrets.
3. **Authorization:** RBAC (patient, doctor, admin) enforced at gateway. Helmet.js headers (HSTS, CSP).
4. **Secrets management:** 16 secrets in Key Vault (RBAC mode); Managed Identity reads at startup—zero passwords in code.
5. **Transport encryption:** TLS everywhere: Front Door HTTPS, PG SSL, Redis 6380/TLS, Cosmos SSL, AMQPS. Rate limiting (50/15min auth, 500 API).

3.4. Scalability & High Availability

Table 2: Scalability mechanisms across the architecture

Component	Mechanism	Details
Container Apps	Horizontal auto-scaling	0→3 replicas based on CPU/memory. Notification Service pinned at min=1.
Cosmos DB	Serverless (auto-RU)	Scales throughput automatically; no capacity planning needed.
PostgreSQL	Burstable B1ms	Can be upgraded to General Purpose or Memory Optimized via Terraform variable.
Front Door	Global CDN	200+ edge locations, automatic origin failover via health probes.
Service Bus	Queue buffering	Messages persist if consumers are down; prevents data loss under load.
Redis	In-memory cache	Reduces PostgreSQL load by caching doctor listings (5-min TTL).

High availability: Cosmos DB provides built-in multi-region replication, PostgreSQL Flexible Server supports zone-redundant HA, Front Door health probes automatically failover between origins, and Container Apps restart failed replicas.

3.5. Database Strategy

A **polyglot persistence** approach matches database technology to data patterns:

- **Cosmos DB (MongoDB API, Serverless)** — 4 databases: user accounts (`auth`), patient records, prescriptions, notifications. Flexible schema + serverless scaling.
- **PostgreSQL 16** — 3 databases: doctor profiles with reviews, appointment bookings with time slots, payment transactions. ACID transactions + foreign key integrity.
- **Redis (Basic C0)** — In-memory cache for Doctor Service. Reduces DB load for read-heavy doctor listing queries (5-min TTL).

3.6. Extensibility

Adding a new microservice (e.g., Lab Results) requires: (1) write service code with Dockerfile, (2) add a `module` block in `compute.tf` reusing the generic container-app module, (3) add a proxy route in the gateway, (4) run `terraform apply`, (5) deploy via existing CI/CD. No existing services need modification.

4. Challenges Faced

Table 3: Major challenges and their resolutions

#	Challenge	Resolution
1	**CORS blocking.** Front Door FQDN not in allowed origins.	Added Front Door + Storage URLs to CORS whitelist; set `followRedirects: true`.
2	**Cosmos DB wire protocol errors.** Mongoose hit unsupported commands on serverless.	Pinned `server_version = "4.2"` in Terraform; added `serverApi` compat.
3	**Redis TLS refused.** Defaulted to non-TLS port 6379.	Set `REDIS_PORT=6380`, `REDIS_TLS=true` in Container App env vars.
4	**SPA deep-link 404s.** Blob Storage returned 404 for `/about`.	Configured Front Door SPA rewrite rule: non-file URLs → `/index.html`.
5	**Email domain not linked to ACS.** Notification Service couldn't send emails.	Added `azurerm_email_communication_service_domain` association in Terraform.
6	**Internal proxy HTTPS loops.** Redirect loops between Container Apps.	Set `allow_insecure = true` on ingress; `secure: false` on proxy middleware.

5. Lessons Learned

1. **IaC is essential for cloud-native development.** Terraform's modular approach let us destroy and recreate the entire 21-resource environment reliably. Manual Azure Portal configuration would be error-prone and non-reproducible.

2. **Database-per-service enforces true microservice boundaries.** Separate databases eliminate hidden coupling—services communicate only through APIs or message queues, making independent scaling and redeployment straightforward.

3. **Async messaging dramatically improves UX.** Appointment booking returns in <200ms because email notifications happen asynchronously via Service Bus, not inline with the API response.

4. **Managed Identity eliminates secrets-in-code.** Passwordless Azure-to-Azure authentication via User-Assigned Managed Identity means no credentials in code or config files.

5. **Cloud services have regional constraints.** Cosmos DB Serverless was unavailable in East US, requiring multi-region deployment (Cosmos in West US 2, PostgreSQL in North Europe)—unintentionally improving resilience.

6. **Selective CI/CD matters for microservices.** Per-service deployment via workflow inputs prevents unnecessary rebuilds. Git SHA image tags enable instant rollback.

7. **CDN + SPA requires explicit routing rules.** Without a catch-all rewrite rule at the Front Door level, single-page application deep links break with 404 errors—a cloud-specific challenge absent in local development.